Zadanie domowe | Lekcja 10 | 31.08.2026

Laboratorium 09 — polecenia do wykonania

Przygotowanie katalogu roboczego

1. Utwórz katalog `work`.
2. Skopiuj katalog `materials/linux-lab` do `work/linux-lab`, zachowując atrybuty plików.
3. Przejdź do `work/linux-lab`.
4. Wyświetl bieżącą ścieżkę.
5. Wyświetl szczegółową listę plików, również ukrytych.

1. Rozpoznanie dystrybucji, kernela i architektury

1. Wyświetl zawartość `/etc/os-release`.
2. Zanotuj wartości `PRETTY_NAME`, `ID`, `VERSION_ID`, jeśli występuje, oraz `VERSION_CODENAME`, jeśli występuje.
3. Sprawdź wersję kernela.
4. Sprawdź architekturę systemu.
5. Sprawdź główną architekturę pakietów.
6. Sprawdź dodatkowe architektury pakietów.
7. Jeśli narzędzie `hostnamectl` jest dostępne, wyświetl informacje o systemie.
8. Odpowiedz, czy wersja uzyskana przez `uname -r` dotyczy dystrybucji, czy kernela.
9. Porównaj nazwę architektury uzyskaną przez `uname -m` z nazwą zwracaną przez `dpkg --print-architecture`.
10. Określ, czy pracujesz na systemie ogólnego przeznaczenia, czy dystrybucji specjalistycznej.
11. Określ, czy środowisko wygląda na maszynę wirtualną, kontener, system live czy pełną instalację.
12. Przygotuj notatkę zawierającą dystrybucję, wydanie lub kryptonim, kernel, architekturę systemu, architekturę pakietów oraz model wydania.

Lp	Wymagana informacja	Komenda	Odpowiedź
1	Zawartość pliku opisującego system	`cat /etc/os-release`	Wyświetlona informacje o Ubuntu.
2	`PRETTY_NAME`, `ID`, `VERSION_ID`, `VERSION_CODENAME`	`cat /etc/os-release`	`PRETTY_NAME="Ubuntu 22.04.5 LTS"`; `ID=ubuntu`; `VERSION_ID="22.04"`; `VERSION_CODENAME=jammy`.

3	Wersja kernela	`uname -r`	`6.8.0-138-generic`
4	Architektura systemu	`uname -m`	`x86_64` — architektura 64-bitowa.
5	Główna architektura pakietów	`dpkg --print-architecture`	`amd64`.
6	Dodatkowe architektury pakietów	`dpkg --print-foreign-architectures`	`i386` — włączona obsługa pakietów 32-bitowych. Nie oznacza to, że takie pakiety są już zainstalowane.
7	Zbiorcze informacje o systemie	`hostnamectl`	Nazwa: `joanna-VirtualBox`; system: Ubuntu 22.04.5 LTS; kernel: `6.8.0-138-generic`; architektura: `x86-64`; wirtualizacja: `oracle`; model sprzętu: `VirtualBox`.
8	Czy `uname -r` pokazuje wersję dystrybucji czy kernela?	`uname -r`	**Kernela.** Ubuntu ma wersję `22.04.5 LTS`, a jego rdzeń — `6.8.0-138-generic`.
9	Porównanie architektury systemu i pakietów	`uname -m` oraz `dpkg --print-architecture`	`x86_64` i `amd64` to **różne nazwy tej samej architektury**. `amd64` działa także na zgodnych procesorach Intel
10	Dystrybucja ogólnego przeznaczenia czy specjalistyczna?	`cat /etc/os-release` — identyfikuje dystrybucję	**Ogólnego przeznaczenia.** Ubuntu służy m.in. do pracy biurowej, programowania i serwerów. Polecenie pokazuje nazwę; klasyfikacja wynika z przeznaczenia Ubuntu.
11	VM, kontener, system live czy pełna instalacja?	`hostnamectl`	**Potwierdzona VM:** `Chassis: vm`, `Virtualization: oracle`, `Hardware Model: VirtualBox`. Sam wynik nie potwierdza, czy Ubuntu jest zainstalowane na wirtualnym dysku, czy uruchomione jako live.

12	Notatka: dystrybucja, wydanie, kernel, architektury i model wydania	Wyniki powyższych komend	Gotowa notatka poniżej

W punkcie 11 ważna jest różnica: **VM opisuje „komputer”, a live lub instalacja opisuje sposób uruchomienia systemu**. Można mieć pełną instalację Ubuntu wewnątrz VM. Aby dodatkowo sprawdzić, skąd działa system, służy:

findmnt /

Pokazuje źródło głównego systemu plików. Wyniku tego polecenia **jeszcze nie mamy**; źródło typu `/dev/sda2` wskazywałoby na instalację na dysku, a `overlay` wymagałoby dalszego sprawdzenia pod kątem live lub kontenera.

Notatka do punktu 12:

> Pracuję na Ubuntu 22.04.5 LTS, dystrybucji ogólnego przeznaczenia. Identyfikator systemu to `ubuntu`, wersja wydania to `22.04`, a kryptonim to `jammy` (Jammy Jellyfish). Kernel ma wersję `6.8.0-138-generic`. Architektura systemu to `x86_64`, a główna architektura pakietów to `amd64` — obie nazwy oznaczają tę samą architekturę 64-bitową. Włączona jest również dodatkowa architektura pakietów `i386`. System działa w maszynie wirtualnej VirtualBox. Ubuntu stosuje model wydań wersjonowanych; LTS oznacza długoterminowe wsparcie. Aktualizacje w ramach wydania nie zmieniają go automatycznie na kolejne wydanie Ubuntu.

Modelu wydania nie ustaliliśmy osobną komendą — `/etc/os-release` identyfikuje Ubuntu i oznaczenie LTS, a opis modelu wynika z zasad wydawania tej dystrybucji.

2. Terminal, powłoka i składnia polecenia

1. Sprawdź urządzenie terminala.
2. Sprawdź rozmiar terminala.
3. Wyświetl wartość zmiennej `TERM`.
4. Wyświetl wartość zmiennej `SHELL`.
5. Sprawdź nazwę procesu aktualnej powłoki.
6. Sprawdź typ polecenia `cd`.
7. Sprawdź typ polecenia `grep`.
8. Znajdź ścieżkę programu `grep`.
9. Wyświetl wartość zmiennej `PATH`.
10. Uruchom polecenie `true` i wyświetl jego kod zakończenia z etykietą „Kod po true”.
11. Uruchom polecenie `false` i wyświetl jego kod zakończenia z etykietą „Kod po false”.
12. Wyszukaj tekst `ERROR` w `logs/application.log` i wyświetl kod zakończenia z etykietą „Kod po grep z trafieniem”.

13. Wyszukaj tekst `NIE_MA_TAKIEGO_TEKSTU` w `logs/application.log` i wyświetl kod zakończenia z etykietą „Kod po grep bez trafienia”.
14. Odpowiedz, czy brak dopasowania w programie `grep` oznacza awarię programu.

Lp	Zadanie	Komenda	Wynik
1	Sprawdzenie urządzenia terminala	`tty`	`/dev/pts/0`
2	Sprawdzenie rozmiaru terminala	`stty size`	`24 80` — 24 wiersze, 80 kolumn
3	Wyświetlenie zmiennej `TERM`	`echo "$TERM"`	`xterm-256color`
4	Wyświetlenie zmiennej `SHELL`	`echo "$SHELL"`	`/bin/bash`
5	Sprawdzenie nazwy procesu aktualnej powłoki	`ps -p $$ -o comm=`	`bash`
6	Sprawdzenie typu polecenia `cd`	`type cd`	Wewnętrzne polecenie powłoki
7	Sprawdzenie typu polecenia `grep`	`type grep`	Alias do `grep --color=auto`
8	Znalezienie ścieżki programu `grep`	`type -P grep`	`/usr/bin/grep`
9	Wyświetlenie zmiennej `PATH`	`echo "$PATH"`	Lista katalogów podana poniżej
10	Uruchomienie `true` i sprawdzenie kodu zakończenia	`true; echo "Kod po true: $?"`	`Kod po true: 0`
11	Uruchomienie `false` i sprawdzenie kodu zakończenia	`false; echo "Kod po false: $?"`	`Kod po false: 1`
12	Wyszukanie `ERROR` w logu i sprawdzenie kodu zakończenia	`grep "ERROR" logs/application.log; echo "Kod po grep z trafieniem: $?"`	Dwa pasujące wiersze; `Kod po grep z trafieniem: 0`

13	Wyszukanie nieobecnego tekstu i sprawdzenie kodu zakończenia	`grep "NIE_MA_TAKIEGO_TEKSTU" logs/application.log; echo "Kod po grep bez trafienia: $?"`	Brak pasujących wierszy; `Kod po grep bez trafienia: 1`
14	Czy brak dopasowania w `grep` oznacza awarię programu?	—	Nie. Kod `1` oznacza brak dopasowania, a nie awarię programu.

Wartość `PATH` — punkt 9:

/home/joanna/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin

Znalezione wiersze — punkt 12:

2026-07-18T08:12:43Z ERROR app=portal component=db message="connection timeout" retry=1
2026-07-18T08:47:55Z ERROR app=portal src=198.51.100.23 action=request path=/api/debug status=404

3. Poruszanie się po systemie plików

1. Wyświetl bieżącą ścieżkę.
2. Wyświetl podstawową listę plików.
3. Wyświetl szczegółową listę plików, również ukrytych.
4. Wyświetl szczegółową listę plików z czytelnymi rozmiarami.
5. Przejdź do katalogu `logs`.
6. Wyświetl bieżącą ścieżkę i szczegółową listę plików, również ukrytych.
7. Wróć do katalogu nadrzędnego i wyświetl bieżącą ścieżkę.
8. Przejdź do katalogu `reports` i wyświetl bieżącą ścieżkę.
9. Wróć do poprzedniego katalogu i ponownie wyświetl bieżącą ścieżkę.
10. Wyjaśnij różnicę między przykładową ścieżką bezwzględną `/home/student/lab`, ścieżką `home/student/lab` oraz ścieżkami względnymi `./logs` i `../reports`.
11. Porównaj zwykłą listę plików ze szczegółową listą uwzględniającą pliki ukryte.
12. Otwórz plik `.analyst-note` w przeglądarce tekstowej.

Punkt	Czynność / polecenie	Wynik
1	`pwd`	`/home/joanna/work/linux-lab`

2	`ls`	`archive-source`, `config`, `downloads`, `logs`, `README.txt`, `reports`, `users.csv`
3	`ls -la`	Szczegółowa lista zawierająca także ukryty plik `.analyst-note` oraz wpisy `.` i `..`.
4	`ls -lh`	Szczegółowa lista z czytelnymi rozmiarami: katalogi `4,0K`, `README.txt` — 570 B, `users.csv` — 348 B.
5	`cd logs`	Przejście do katalogu `logs`.
6	`pwd` oraz `ls -la`	Ścieżka: `/home/joanna/work/linux-lab/logs`. Pliki: `application.log` — 1023 B, `auth.log` — 1075 B, `old-application.log` — 400 B.
7	`cd ..`, następnie `pwd`	Powrót do katalogu nadrzędnego: `/home/joanna/work/linux-lab`.
8	`cd reports`, następnie `pwd`	`/home/joanna/work/linux-lab/reports`
9	`cd -`, następnie `pwd`	`/home/joanna` — poprzednio odwiedzony katalog w nowej sesji terminala po restarcie Ubuntu.
10	Wyjaśnienie ścieżek	Ścieżka zaczynająca się od `/` jest bezwzględna. Ścieżka bez początkowego `/` jest względna wobec bieżącego katalogu.
11	Porównanie `ls` i `ls -la`	`ls` pokazuje nazwy widocznych plików i katalogów. `ls -la` pokazuje również pliki ukryte i szczegóły wpisów.
12	`less .analyst-note`; wyjście klawiszem `q`	Odczytano: „Ukryta notatka: Sprawdz, czy student pamieta o ls -la.”

Znaczenie przykładowych ścieżek:

Ścieżka	Znaczenie
`/home/student/lab`	Bezwzględna — liczona od głównego katalogu systemu `/`.
`home/student/lab`	Względna — liczona od bieżącego katalogu.
`./logs`	Katalog `logs` wewnątrz bieżącego katalogu.

`../reports`	Katalog `reports` wewnątrz katalogu nadrzędnego.

4. Tworzenie katalogu sprawy i praca na kopii

1. Utwórz strukturę `case-001/evidence`, `case-001/notes` i `case-001/output`.
2. Wyświetl rekurencyjnie strukturę katalogu `case-001`.
3. Skopiuj katalog `logs` do `case-001/evidence/`, zachowując atrybuty.
4. Skopiuj katalog `reports` do `case-001/evidence/`, zachowując atrybuty.
5. Skopiuj plik `users.csv` do `case-001/evidence/`, zachowując atrybuty.
6. Sprawdź typ pliku `case-001/evidence/logs/application.log`.
7. Sprawdź typ pliku `case-001/evidence/users.csv`.
8. Sprawdź typ pliku `case-001/evidence/reports/report-2026.txt`.
9. Utwórz pusty plik `case-001/notes/timeline.txt`.
10. Wyświetl szczegółowe informacje o utworzonym pliku.

Zadanie	Komenda	Wynik
1. Utworzenie struktury katalogów	`mkdir -p case-001/{evidence,notes,output}`	Utworzono katalogi `evidence`, `notes` i `output`.
2. Wyświetlenie struktury katalogów	`ls -R case-001`	Wyświetlono strukturę katalogów i podkatalogów.
3. Kopiowanie katalogu `logs`	`cp -a logs case-001/evidence/`	Skopiowano katalog `logs` z zachowaniem atrybutów.
4. Kopiowanie katalogu `reports`	`cp -a reports case-001/evidence/`	Skopiowano katalog `reports` z zachowaniem atrybutów.
5. Kopiowanie pliku `users.csv`	`cp -a users.csv case-001/evidence/`	Skopiowano plik `users.csv` z zachowaniem atrybutów.
6. Sprawdzenie typu `application.log`	`file case-001/evidence/logs/application.log`	`ASCII text`
7. Sprawdzenie typu `users.csv`	`file case-001/evidence/users.csv`	`CSV text`
8. Sprawdzenie typu `report-2026.txt`	`file case-001/evidence/reports/report-2026.txt`	`ASCII text`

9. Utworzenie pustego pliku `timeline.txt`	`touch case-001/notes/timeline.txt`	Utworzono pusty plik.
10. Wyświetlenie informacji o pliku	`ls -l case-001/notes/timeline.txt`	`-rw-rw-r-- 1 joanna joanna 0 paź 12 11:14 case-001/notes/timeline.txt`

5. Czytanie logów bez utraty kontroli

1. Wyświetl rozmiary `logs/application.log` i `logs/auth.log` w czytelnej postaci.
2. Sprawdź typ obu plików.
3. Policz wiersze w obu plikach.
4. Wyświetl pierwszych pięć wierszy `logs/application.log`.
5. Wyświetl ostatnich pięć wierszy `logs/application.log`.
6. Otwórz `logs/application.log` w programie `less` bez zawijania długich wierszy.
7. Wyszukaj w otwartym pliku tekst `ERROR`.
8. Przejdź do następnego dopasowania.
9. Przejdź na koniec pliku.
10. Wróć na początek pliku.
11. Zamknij program `less`.
12. Odpowiedz, dlaczego `less` jest bezpieczniejsze niż `cat` dla dużych plików.
13. Odpowiedz, co może się stać po wyświetleniu pliku binarnego w terminalu.

Zadanie	Komenda	Wynik
1. Wyświetlenie rozmiarów plików	`ls -lh logs/application.log logs/auth.log`	`application.log` — 1023 B; `auth.log` — 1,1 KB
2. Sprawdzenie typów plików	`file logs/application.log logs/auth.log`	Oba pliki: `ASCII text`
3. Policzenie wierszy	`wc -l logs/application.log logs/auth.log`	`application.log` — 12; `auth.log` — 10; razem — 22

4. Wyświetlenie pierwszych 5 wierszy	`head -n 5 logs/application.log`	Wyświetlono 5 pierwszych wierszy, m.in. `INFO`, `WARN` i `ERROR`.
5. Wyświetlenie ostatnich 5 wierszy	`tail -n 5 logs/application.log`	Wyświetlono 5 ostatnich wierszy, m.in. błąd `404`, zablokowane przesłanie pliku i zatrzymanie aplikacji.
6. Otwarcie logu bez zawijania wierszy	`less -S logs/application.log`	Otworzono plik w `less`; długie wiersze nie są zawijane.
7. Wyszukanie `ERROR`	`/ERROR` (w `less`)	Wyszukano wystąpienie tekstu `ERROR`.
8. Przejście do następnego dopasowania	`n` (w `less`)	Przejście do następnego wystąpienia `ERROR`.
9. Przejście na koniec pliku	`G` (w `less`)	Przeniesiono widok na koniec pliku.
10. Powrót na początek pliku	`g` (w `less`)	Przeniesiono widok na początek pliku.
11. Zamknięcie `less`	`q` (w `less`)	Zamknięto program i wrócono do terminala.
12. Dlaczego `less` jest bezpieczniejsze niż `cat` dla dużych plików?	—	Wyświetla zawartość fragmentami, umożliwia wyszukiwanie i nawigację, nie zalewając terminala całym plikiem.
13. Co może się stać po wyświetleniu pliku binarnego w terminalu?	—	Mogą pojawić się nieczytelne znaki lub może dojść do zakłócenia wyświetlania terminala.

6. Wyszukiwanie informacji przez grep

1. Znajdź tekst `ERROR` w `logs/application.log`.
2. Powtórz wyszukiwanie z numerami wierszy.
3. Znajdź tekst `failed` w `logs/auth.log` bez rozróżniania wielkości liter.
4. Powtórz wyszukiwanie z numerami wierszy.
5. Policz dopasowania tekstu `failed` w `logs/auth.log`, nie rozróżniając wielkości liter.
6. Rekurencyjnie znajdź aktywność adresu `198.51.100.23` w bieżącym katalogu, pokazując nazwy plików i numery wierszy.

7. W ten sam sposób znajdź aktywność adresu `203.0.113.44`.
8. Wyświetl `config/service.conf` z pominięciem wierszy zaczynających się znakiem komentarza.
9. W ten sam sposób wyświetl `config/example.sources`.
10. W trybie cichym wyszukaj `198.51.100.23` w `logs/auth.log` i wyświetl kod z etykietą „Kod po trafieniu”.
11. W trybie cichym wyszukaj `10.10.10.10` w `logs/auth.log` i wyświetl kod z etykietą „Kod po braku trafienia”.
12. Użyj niepoprawnego wzorca `[` podczas przeszukiwania `logs/auth.log` i wyświetl kod z etykietą „Kod po blednym wzorcu”.
13. Ustal, które kody oznaczają trafienie, brak trafienia i błąd.

Zadanie	Komenda	Wynik
1. Wyszukanie błędów	`grep 'ERROR' logs/application.log`	Znaleziono 2 linie zawierające `ERROR`.
2. Wyświetlenie numerów linii	`grep -n 'ERROR' logs/application.log`	Błędy znajdują się w liniach 4 i 8.
3. Wyszukanie nieudanych prób	`grep -i 'failed' logs/auth.log`	Znaleziono 5 linii zawierających `failed`, niezależnie od wielkości liter.
4. Wyszukanie z numerami linii	`grep -in 'failed' logs/auth.log`	Znaleziono wpisy w liniach 2, 3, 6, 9 i 10.
5. Zliczenie dopasowań	`grep -ic 'failed' logs/auth.log`	Wynik: `5` — pięć linii zawiera szukany tekst.
6. Wyszukanie adresu IP	`grep -rn '198.51.100.23' .`	Znaleziono adres w logach, raporcie i osi czasu, również w kopiach w `case-001/evidence`.
7. Wyszukanie drugiego adresu IP	`grep -rn '203.0.113.44' .`	Znaleziono adres w logach uwierzytelniania, starszym logu aplikacji i osi czasu, również w kopiach.
8. Pominięcie komentarzy w konfiguracji	`grep -v '^#' config/service.conf`	Wyświetlono ustawienia usługi bez linii zaczynających się od `#`.
9. Pominięcie komentarzy w źródłach pakietów	`grep -v '^#' config/example.sources`	Wyświetlono ustawienia źródła pakietów Debiana bez linii komentarzy.

10. Sprawdzenie istniejącego adresu	`grep -q '198.51.100.23' logs/auth.log; echo "Kod po trafieniu: $?"`	Wynik: `0` — znaleziono dopasowanie.
11. Sprawdzenie nieistniejącego adresu	`grep -q '10.10.10.10' logs/auth.log; echo "Kod po braku trafienia: $?"`	Wynik: `1` — nie znaleziono dopasowania.
12. Sprawdzenie błędnego wzorca	`grep -q '[' logs/auth.log; echo "Kod po blednym wzorcu: $?"`	Wynik: `2` — błąd wyrażenia regularnego.
13. Interpretacja kodów zakończenia	—	`0` — znaleziono dopasowanie; `1` — brak dopasowania; `2` — błąd.

Najważniejsze opcje `grep`

Opcja / symbol	Znaczenie
`-i`	Ignoruje wielkość liter.
`-n`	Wyświetla numery linii.
`-c`	Zlicza linie zawierające dopasowanie.
`-r`	Wyszukuje rekurencyjnie w katalogach i podkatalogach.
`-v`	Wyświetla linie, które nie pasują do wzorca.
`-q`	Wykonuje wyszukiwanie bez wyświetlania dopasowanych linii.
`^#`	Wzorzec oznaczający znak `#` na początku linii.
`.`	Bieżący katalog.
`$?`	Kod zakończenia poprzedniego polecenia.

Wniosek z laboratorium: `grep` pozwala szybko wyszukiwać konkretne informacje w logach i plikach konfiguracyjnych. W analizie bezpieczeństwa przydaje się do identyfikowania błędów, nieudanych prób logowania i wystąpień podejrzanych adresów IP.

7. Wyszukiwanie plików przez find

1. Znajdź wszystkie zwykłe pliki w bieżącym katalogu i jego podkatalogach.
2. Znajdź zwykłe pliki o nazwach kończących się na `.log`.
3. Znajdź zwykłe pliki zawierające w nazwie słowo `report`, bez rozróżniania wielkości liter.
4. Znajdź zwykłe pliki zmodyfikowane w ostatniej dobie.
5. Wyświetl listę wszystkich znalezionych logów.
6. Dla każdego znalezionego logu sprawdź typ pliku.

Zadanie	Komenda	Wynik
1. Znajdź wszystkie zwykłe pliki	`find . -type f`	Wyświetlono zwykłe pliki w bieżącym katalogu i podkatalogach, w tym pliki ukryte.
2. Znajdź pliki `.log`	`find . -type f -name "*.log"`	Znaleziono 6 plików logów: 3 oryginalne i 3 kopie.
3. Znajdź pliki z `report` w nazwie	`find . -type f -iname "*report*"`	Znaleziono 4 pliki raportów z lat 2025 i 2026 oraz ich kopie.
4. Znajdź pliki zmodyfikowane w ostatniej dobie	`find . -type f -mtime -1`	Znaleziono `./case-001/notes/timeline.txt`.
5. Wyświetl wszystkie logi	`find . -type f -name "*.log"`	Ponownie wyświetlono listę 6 plików `.log`.
6. Sprawdź typ każdego logu	`find . -type f -name "*.log" -exec file {} \;`	Wszystkie 6 plików rozpoznano jako `ASCII text`.

8. Potoki i przekierowania

1. Zapisz nieudane logowania z `logs/auth.log` do `case-001/output/failed-logins.txt`, nie rozróżniając wielkości liter.
2. Policz wiersze w zapisanym pliku i dopisz wynik do `case-001/output/summary.txt`.
3. Wyszukaj w `/etc` pliki o nazwach kończących się na `.conf`; zapisz wyniki do `case-001/output/etc-conf-files.txt`, a błędy do `case-001/output/etc-errors.txt`.
4. Powtórz wyszukiwanie, zapisując wynik i błędy razem w `case-001/output/etc-combined.txt`.
5. Wykonaj przekierowanie wyniku do `case-001/output/order-a.txt`, a następnie skieruj błędy do tego samego miejsca.
6. Powtórz operację w odwrotnej kolejności i zapisz wynik do `case-001/output/order-b.txt`.
7. Porównaj rezultaty obu kolejności przekierowań.

8. Wyszukaj nieudane logowania, zapisz wynik pośredni do `case-001/output/failed-with-tee.txt` i jednocześnie policz wiersze.
9. Spróbuj wyszukać nieudane logowania w nieistniejącym pliku `missing.log`, przekazując wynik do liczenia wierszy.
10. W powłoce Bash wyświetl statusy wszystkich elementów ostatniego potoku.

Zadanie	Komenda	Wynik
1. Zapis nieudanych logowań	`grep -i 'failed' logs/auth.log > case-001/output/failed-logins.txt`	Zapisano 5 pasujących wierszy do pliku.
2. Policz wiersze i dopisz wynik	`wc -l case-001/output/failed-logins.txt >> case-001/output/summary.txt`	Wynik liczenia dopisano do `summary.txt`.
3. Zapisz pliki `.conf` i błędy osobno	`find /etc -type f -name "*.conf" > case-001/output/etc-conf-files.txt 2> case-001/output/etc-errors.txt`	Wyniki zapisano do `etc-conf-files.txt`, błędy do `etc-errors.txt`.
4. Zapisz wyniki i błędy razem	`find /etc -type f -name "*.conf" > case-001/output/etc-combined.txt 2>&1`	Standardowe wyjście i błędy skierowano do jednego pliku.
5. Najpierw przekieruj wynik	`find /etc -type f -name "*.conf" > case-001/output/order-a.txt 2>&1`	Wyniki i błędy zapisano w `order-a.txt`.
6. Odwróć kolejność	`find /etc -type f -name "*.conf" 2>&1 > case-001/output/order-b.txt`	Wyniki zapisano w `order-b.txt`, a błędy wyświetliły się w terminalu.
7. Porównaj pliki	`wc -l case-001/output/order-a.txt case-001/output/order-b.txt`	`order-a.txt`: 354 wiersze; `order-b.txt`: 351 wierszy.
8. Zapisz wynik i policz wiersze	`grep -i 'failed' logs/auth.log | tee`	Zapisano wpisy do pliku i wyświetlono wynik 5.

	`case-001/output/failed-with-te e.txt | wc -l`	
9. Potok z nieistniejącym plikiem	`grep -i 'failed' missing.log | wc -l`	`grep` zgłosił błąd, a `wc -l` wyświetlił `0`.
10. Sprawdź statusy potoku	`grep -i 'failed' missing.log | wc -l; echo "Statusy potoku: ${PIPESTATUS[*]}"`	Statusy: `2 0` — błąd `grep`, poprawne wykonanie `wc`.

9. Edycja notatki w Nano

1. Otwórz `case-001/notes/timeline.txt` w edytorze Nano.
2. Dodaj wpis o serii nieudanych logowań dla użytkownika `admin` z adresu `203.0.113.44` o godzinie 08:05.
3. Dodaj wpis o nieudanym logowaniu użytkownika `root` z adresu `198.51.100.23` o godzinie 08:21.
4. Dodaj wpis o odmowie wykonania `sudo` przez użytkownika `jan.zielinski` o godzinie 08:42.
5. Zapisz plik, zatwierdź jego nazwę i zamknij edytor.
6. Wyświetl zawartość `case-001/notes/timeline.txt`.

Zadanie	Komenda / czynność	Wynik
1. Otwarcie pliku	`nano case-001/notes/timeline.txt`	Otworzono plik w edytorze Nano.
2. Wpis o użytkowniku admin	Wpisano tekst w Nano	Dodano informację o serii nieudanych logowań z `203.0.113.44` o 08:05.
3. Wpis o użytkowniku root	Wpisano tekst w Nano	Dodano informację o nieudanym logowaniu z `198.51.100.23` o 08:21.
4. Wpis o `sudo`	Wpisano tekst w Nano	Dodano informację o odmowie wykonania `sudo` przez `jan.zielinski` o 08:42.
5. Zapis i zamknięcie	`Ctrl + O`, Enter, `Ctrl + X`	Zapisano plik i zamknięto edytor.
6. Weryfikacja zawartości	`cat case-001/notes/timeline.txt`	Wyświetlono trzy zapisane wpisy.

10. Minimum Vim

1. Otwórz `case-001/notes/timeline.txt` w edytorze Vim.
2. Przejdź do trybu wstawiania.
3. Dopisz wniosek, że widoczne są próbne zdarzenia brute force.
4. Wróć do trybu normalnego.
5. Zapisz plik i zamknij edytor.
6. Otwórz ten sam plik ponownie.
7. Przejdź do trybu wstawiania i dopisz linię testową.
8. Wróć do trybu normalnego.
9. Zamknij edytor bez zapisywania zmian.
10. Wyświetl plik i sprawdź, czy linia testowa nie została zapisana.

Operacja	Polecenie / klawisz
Otworzenie pliku	`vim case-001/notes/timeline.txt`
Przejście do ostatniej linii	`G`
Dodanie nowej linii i rozpoczęcie pisania	`o`
Powrót do trybu normalnego	`Esc`
Zapisanie i zamknięcie	`:wq`
Wyjście bez zapisywania	`:q!`
Wyświetlenie pliku w terminalu	`cat case-001/notes/timeline.txt`

11. Tworzenie i sprawdzanie archiwum

1. Utwórz skompresowane archiwum `case-001.tar.gz` zawierające katalog `case-001/`.
2. Sprawdź typ archiwum.
3. Wyświetl jego zawartość w programie `less` bez rozpakowywania.
4. Oblicz sumę SHA-256 archiwum i zapisz wynik w `case-001.tar.gz.sha256`.
5. Wyświetl zapisany plik sumy kontrolnej.
6. Utwórz katalog `extracted/case-001-test`.
7. Rozpakuj archiwum do tego katalogu.
8. Znajdź wszystkie zwykłe pliki w rozpakowanej strukturze.

Zadanie	Polecenie
1. Utworzenie archiwum	`tar -czf case-001.tar.gz case-001/`
2. Sprawdzenie typu	`file case-001.tar.gz`
3. Podgląd zawartości bez rozpakowania	`tar -tzf case-001.tar.gz | less`
4. Obliczenie i zapis SHA-256	`sha256sum case-001.tar.gz > case-001.tar.gz.sha256`
5. Wyświetlenie sumy	`cat case-001.tar.gz.sha256`
6. Utworzenie katalogu testowego	`mkdir -p extracted/case-001-test`
7. Rozpakowanie archiwum	`tar -xzf case-001.tar.gz -C extracted/case-001-test`
8. Znalezienie zwykłych plików	`find extracted/case-001-test -type f`

12. Pobieranie danych przez curl i wget

1. Wyświetl nagłówki odpowiedzi adresu `https://example.com/`.
2. Pobierz ten adres przez `curl` do `downloads/example-curl.html`.
3. Pobierz ten sam adres przez `wget` do `downloads/example-wget.html`.
4. Sprawdź typ obu pobranych plików.
5. Wyświetl szczegółową listę katalogu `downloads/` z czytelnymi rozmiarami.
6. Wyświetl pierwszych pięć wierszy `downloads/example-curl.html`.
7. Oblicz sumy SHA-256 obu pobranych plików.
8. Nie uruchamiaj pobranych plików.
9. Wyjaśnij kolejność działań: pobranie, sprawdzenie, przeczytanie, zweryfikowanie i dopiero potem rozważenie uruchomienia.
10. Nie pobieraj skryptu instalacyjnego bezpośrednio do powłoki.

Zadanie	Komenda	Wynik
1. Wyświetlenie nagłówków odpowiedzi	`curl -I https://example.com/`	Nagłówki HTTP serwera, np. kod statusu `200 OK`
2. Utworzenie katalogu na pobrane pliki	`mkdir -p downloads`	Utworzony katalog `downloads/` (jeśli wcześniej nie istniał)

3. Pobranie strony przez `curl`	`curl https://example.com/ -o downloads/example-curl.html`	Strona zapisana w pliku `example-curl.html`
4. Pobranie strony przez `wget`	`wget https://example.com/ -O downloads/example-wget.html`	Strona zapisana w pliku `example-wget.html`
5. Sprawdzenie typu plików	`file downloads/example-curl.html downloads/example-wget.html`	Typ obu pobranych plików
6. Wyświetlenie szczegółów katalogu	`ls -lh downloads/`	Lista plików, uprawnienia, daty i czytelne rozmiary
7. Wyświetlenie pierwszych 5 wierszy	`head -n 5 downloads/example-curl.html`	Pierwszych pięć wierszy zawartości HTML
8. Obliczenie sum SHA-256	`sha256sum downloads/example-curl.html downloads/example-wget.html`	Dwie sumy kontrolne SHA-256
9. Bezpieczna kolejność działań	Brak komendy — zadanie teoretyczne	Pobranie → sprawdzenie → przeczytanie → zweryfikowanie → rozważenie uruchomienia
10. Nie uruchamiać pobranych plików	Nie uruchamialiśmy plików	Pliki pozostały zapisane na dysku i nie zostały wykonane

suma kontrolna:

```
joanna@joanna-VirtualBox:~/work/linux-lab$ sha256sum downloads/example-curl.html
 downloads/example-wget.html
25ddf2c883e0d1958ea971d279a7e4f0fd446724ee3db7db19dadabd4a62e484  downloads/exam
ple-curl.html
25ddf2c883e0d1958ea971d279a7e4f0fd446724ee3db7db19dadabd4a62e484  downloads/exam
ple-wget.html
```

13. Pakiety i repozytoria bez instalowania

1. Sprawdź, czy program `apt` jest dostępny.
2. Wyszukaj pakiet `jq`.
3. Wyświetl informacje o pakiecie `jq`.
4. Sprawdź politykę i źródła wersji pakietu `jq`.
5. Wyświetl listę zainstalowanych pakietów w programie `less`.
6. Wyświetl w programie `less` pakiety oznaczone jako zainstalowane ręcznie.
7. Wyświetl pakiety wstrzymane.
8. Rekurencyjnie wyświetl niezakomentowane wpisy z `/etc/apt/sources.list` i `/etc/apt/sources.list.d/`, pokazując numery wierszy i ukrywając komunikaty o błędach.
9. Wyświetl ogólną politykę źródeł pakietów.
10. Odpowiedz, co robi `apt update`, a czego nie robi.
11. Wyjaśnij, dlaczego `apt upgrade` zmienia system.
12. Wyjaśnij, dlaczego zewnętrzne repozytorium rozszerza granicę zaufania.
13. Wyjaśnij, dlaczego sam numer wersji programu nie zawsze wystarcza do potwierdzenia podatności.

Zadanie	Komenda	Wynik
1. Sprawdzenie dostępności `apt`	`command -v apt`	Ścieżka do programu, np. `/usr/bin/apt`
2. Wyszukanie pakietu `jq`	`apt search jq`	Lista pakietów pasujących do wyszukiwania
3. Informacje o pakiecie `jq`	`apt show jq`	Wersja, opis, zależności i źródło pakietu
4. Polityka wersji pakietu `jq`	`apt-cache policy jq`	Wersja zainstalowana, kandydat do instalacji i dostępne źródła
5. Lista zainstalowanych pakietów	`apt list --installed | less`	Lista zainstalowanych pakietów
6. Pakiety oznaczone jako	`apt-mark showmanual | less`	Lista pakietów oznaczonych jako instalowane ręcznie

zainstalowane ręcznie		
7. Pakiety wstrzymane	`apt-mark showhold`	Lista wstrzymanych pakietów lub pusty wynik
8. Aktywne wpisy repozytoriów	`grep -rnE '^[[:space:]]*[^#[:space:]]' /etc/apt/sources.list /etc/apt/sources.list.d/ 2>/dev/null`	Niezakomentowane wpisy z numerami wierszy
9. Ogólna polityka źródeł	`apt-cache policy`	Lista źródeł pakietów i ich priorytetów
10. Działanie `apt update`	Zadanie teoretyczne	Aktualizuje informacje o dostępnych pakietach, nie aktualizuje programów
11. Działanie `apt upgrade`	Zadanie teoretyczne	Instaluje aktualizacje pakietów i zmienia stan systemu
12. Ryzyko zewnętrznych repozytoriów	Zadanie teoretyczne	Rozszerzają granicę zaufania o dodatkowego dostawcę oprogramowania
13. Numer wersji a podatności	Zadanie teoretyczne	Starsza wersja może zawierać poprawki bezpieczeństwa dzięki backportingowi

Wniosek: Polecenia diagnostyczne APT pozwalają sprawdzać pakiety, wersje i repozytoria bez instalowania oprogramowania. `apt update` odświeża informacje o pakietach, natomiast `apt upgrade` aktualizuje zainstalowane oprogramowanie.

14. Zadanie końcowe

1. Umieść kopie logów, raportów i `users.csv` w `case-001/evidence/`.
2. Umieść linie nieudanych logowań w `case-001/output/failed-logins.txt`.
3. Umieść liczbę nieudanych logowań w `case-001/output/summary.txt`.
4. Umieść trzy najważniejsze obserwacje w `case-001/notes/timeline.txt`.
5. Utwórz `case-001.tar.gz` zawierające katalog `case-001/`.
6. Zapisz hash archiwum w `case-001.tar.gz.sha256`.
7. Wyświetl wszystkie zwykłe pliki w `case-001`.

8. Wyświetl zawartość `case-001.tar.gz` bez rozpakowywania.
9. Zweryfikuj sumę kontrolną zapisaną w `case-001.tar.gz.sha256`.

Zadanie	Komenda	Wynik
1. Kopie logów, raportów i `users.csv`	`ls -R case-001/evidence/`	Pliki dowodowe są w katalogu `evidence/`
2. Linie nieudanych logowań	`cat case-001/output/failed-logins.txt`	5 nieudanych logowań
3. Liczba nieudanych logowań	`cat case-001/output/summary.txt`	Liczba 5
4. Obserwacje i wniosek	`cat case-001/notes/timeline.txt`	Oś czasu zawiera obserwacje i wniosek
5. Utworzenie archiwum	`tar -czf case-001.tar.gz case-001/`	Utworzone archiwum `.tar.gz`
6. Zapisanie sumy SHA-256	`sha256sum case-001.tar.gz > case-001.tar.gz.sha256`	Suma zapisana w pliku
7. Lista zwykłych plików	`find case-001 -type f`	Lista plików w strukturze sprawy
8. Podgląd archiwum	`tar -tzf case-001.tar.gz | less`	Zawartość wyświetlona bez rozpakowywania
9. Weryfikacja sumy	`sha256sum -c case-001.tar.gz.sha256`	`case-001.tar.gz: OK`

```
joanna@joanna-VirtualBox:~/work/linux-lab$ tar -tzf case-001.tar.gz | less
joanna@joanna-VirtualBox:~/work/linux-lab$ sha256sum -c case-001.tar.gz.sha256
case-001.tar.gz: DOBRZE
joanna@joanna-VirtualBox:~/work/linux-lab$ sha256sum -c case-001.tar.gz.sha256
case-001.tar.gz: DOBRZE
joanna@joanna-VirtualBox:~/work/linux-lab$
```